

Document Title: MarketMinds UI and Functionality Breakdown

Designed by: Ishpreet Singh, Joel Lee, and Wille Gadson

Screen 1: Live Watchlist Dashboard

Primary Purpose: The main purpose of this screen is it acts like the main landing dashboard for real-time market tracking, which we optimized it for fast data ingestion so it load quick.

- **Real-Time Price Fetching:** This screen acts as the UI layer for the continuous API background requests and it displaying the most current stock prices.
- **Asynchronous UI Updates:** It features a visual update flash state that highlight the price text the exact millisecond when new data arrive from the backend server, and this prevent the users screen from freezing or requiring a manual page refresh which can be very annoying for them.
- **Lightweight Data Rendering:** We intentionally restricted it to only display the Symbol, Company Name and the Price because this strict functionality make sure the JSON payload from the server stay incredibly small, resulting in zero lag when it rendering.

Screen 2: Stock Detail and Chart View

Primary Purpose: The primary purpose is visualizing historical market trends for one single selected asset that the user clicked on.

- **Historical Data Parsing:** It take the historical time-series data array which have timestamps and daily close prices fetched by the server, and parse it for the front-end rendering.
- **Changing Line Graphs:** It generates a continuous smooth Canvas line graph where the UI mathematically scale the X-axis for time and the Y-axis for price limits dynamically based on the users desktop viewport width so it look correct.
- **Optimized Load Speed:** It deliberately exclude secondary brokerage metadatas like 52-week highs and volume, this is to ensure the chart render instantly without having to wait for unbilled secondary API requests which would slow down the whole application.

Screen 3: Options Pricing Engine (The Calculator)

Primary Purpose: The main purpose is handling the complex user inputs and to visualize the multi-core mathematical processing engine working.

- **Manual Parameter Validation:** It provide dedicated form fields for all the variables required by the Black-Scholes and Monte Carlo models, which are Ticker, Strike Price, Time, Risk-Free Rate, and Volatility, so it ensure the math engine gets the exact parameters needed without triggering an external database searches.
- **Parallel Computing Visualization:** It features a localized Distributing to CPU Cores processing state which specifically handles the latency caused when the backend split the 100,000 simulated Monte Carlo paths across four separate CPU cores which takes a lot of time to calculate.
- **Instant Result Categorization:** Immediately when it recieve the theoretical price from the backend, the UI render the data and executes logic to assign a dynamic status badge categorizing the option as In the Money, Out of the Money, or Knocked Out.

Screen 4: Component-Level Error State

Primary Purpose: The primary purpose is to handle API timeouts and the network disconnections without crashing the application.

- **Localized Failure Rendering:** If the Yahoo Finance API drops, this functionality restrict the error to only the specific panel rather then showing a generic full-page browser error to the user.
- **Clearing the Data:** It automatically overrides the results panel using blank dashes when a connection fail, which is a critical functionality that ensure the application never accidentally displays the cached or outdated financial data.
- **Manual API Retry:** It provides a dedicated trigger button to re-initiate the HTTP GET request to the Python server once the users internet connection are restored back to normal.